LINUX MEMORY · OOM

리눅스 메모리와 OOM Killer

컴퓨터가 메모리를 나눠 주고,
모자라면 누군가를 내보내는 이야기

OOM Killer는 어떤 기준으로 프로세스를 고를까?

01 **메모리 관리 기초**
메모리 빌려주기 · 가상 메모리, Demand Paging

02 **메모리 회수**
꽉 차면 정리하기 · Page Reclaim, Watermark

03 **Page Cache 개요**
버려도 되는 메모리 · Page Cache, rss, working_set

04 **OOM 트리거 경로**
쫓아내기가 시작될 때 · 전역 OOM, memcg OOM, out_of_memory()

05 **oom_badness() 스코어링**
누구를 내보낼까 · oom_badness(), oom_score_adj

06 **OOM dmesg 메시지 해부**
사건 기록부 읽기 · dmesg, 플레이북

먼저 큰 그림

프로그램이 쫓겨나기까지 6단계

이 자료의 모든 슬라이드는 아래 한 칸을 크게 확대한 것입니다

1
자리 예약
번호표만 받음
malloc · mmap

→

2
실제로 쓰기
이제 책상을 차지
page fault · RSS

→

3
꽉 차면 정리
복사본부터 버림
page reclaim

→

4
정리 실패
더 버릴 게 없음
out_of_memory()

→

5
벌점 계산
제일 많이 쓴 쪽
oom_badness()

→

6
강제 퇴장
기록이 남음
SIGKILL · dmesg

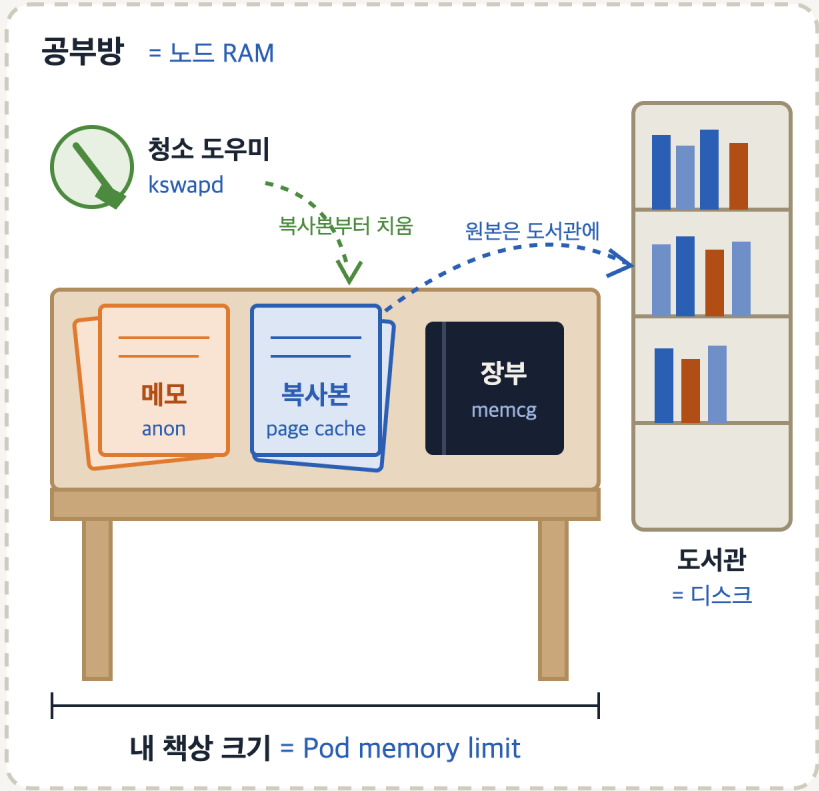
쿠버네티스에서는 6단계가 Pod 상태 **OOMKilled**, 종료 코드 **137**로 보인다

읽는 법 · 01장은 비유로 기초 · 02장부터 커널 개념 중심 · 쿠버네티스 연결은 부록

비유 사전 · 계속 이 그림으로 설명합니다

메모리 = 공부방 책상

비유	진짜 이름	한 줄 뜻
공부방 전체	노드의 RAM	서버 한 대의 메모리 전부
내 책상 크기	Pod memory limit	한 프로그램이 쓸 수 있는 최대치
직접 쓴 메모	익명 메모리 (rss)	원본이 없어서 버리면 끝
도서관 책 복사본	파일 캐시 (Page Cache)	버려도 다시 복사하면 됨
청소 도우미	kswapd	미리미리 복사본을 치워 줌
사용량 장부	cgroup (memcg)	누가 얼마나 쓰는지 세고 막음
벌점	oom_score	높을수록 먼저 쫓겨남
강제 퇴장	OOM Kill	프로그램을 즉시 끝냄
사건 기록부	dmesg (커널 로그)	누가 왜 쫓겨났는지 기록



01 메모리 관리 기초

비유: 메모리 빌려주기

이 장의 질문 · malloc 하면 바로 RAM을 쓸까?

번호표와 사물함 · 가상 주소 | 주소 공간 지도 · 커널 메모리맵 | 예약 vs 사용 · Demand Paging | 프로세스 주소 공간 · /proc/pid/maps | 버릴 수 있나 · 익명 vs 파일

같이 볼 문서

메모리 관리 기초 개념 · minzkn.com/linuxkernel/pages/memory.html#memory-fundamentals
익명 페이지와 파일 기반 페이지 · minzkn.com/linuxkernel/pages/memory.html#anonymous-vs-file-pages
malloc()에서 커널까지 · minzkn.com/linuxkernel/pages/memory.html#malloc-to-kernel
커널 메모리맵 · minzkn.com/linuxkernel/pages/memory.html#kernel-memory-map
VMA / mmap 상세 · minzkn.com/linuxkernel/pages/vma-mmap.html

번호표와 진짜 사물함

프로그램은 진짜 위치를 모른다. 번호표만 들고 있고, 장부가 진짜 칸을 알려 준다



그래서 같은 "3번"이어도 서로의 물건을 볼 수 없다 · 프로세스 격리

컨테이너는 같은 건물을 함께 쓴다 · 벽이 아니라 **사용량 장부**(cgroup)로 나눔

어려운 말로: 번호표 = 가상 주소 · 진짜 칸 = 물리 프레임(4KB) · 장부 = 페이지 테이블 · 장부 보는 직원 = MMU

예약했다고 바로 쓰는 게 아니다

1 · 예약 · malloc() / mmap()

"100칸 써도 돼?"

"OK" · 번호표만 줌
책상은 아직 그대로

→

2 · 처음 손댐 · page fault

"어? 칸이 비었네"

관리인(커널)을 부름
handle_mm_fault()

→

3 · 진짜 배정 · alloc_page()

"여기 한 칸 줄게"

이때부터 책상 차지
memcg에 charge

VSZ (VmSize) = 예약한 가상 크기

식당 예약 10명 · 벌점에 안 들어감

RSS (VmRSS) = 실제 RAM 상주량

실제로 온 4명 · 쫓겨날지 판단하는 기준

어려운 말로: 1 = malloc()/mmap()으로 가상 주소 예약 · 2 = page fault · 3 = 물리 페이지 할당 (Demand Paging)

java 프로세스 하나가 주소 공간을 받는 순서

구간(VMA)은 시작할 때 예약되고, 물리 메모리는 쓸 때 붙는다

① **커널 · execve()**
빈 주소 공간과 페이지 테이블 생성 · 위쪽 커널 영역은 자동으로 붙음

② **커널**
java 실행 파일의 코드·데이터 구간 매핑

③ **커널**
메인 스레드 [stack] 구간

④ **동적 로더 · mmap()**
libjvm.so, libc.so 같은 공유 라이브러리 구간

⑤ **JVM · mmap()**
heap(-Xmx), Metaspace, Code cache, 스레드 스택 구간 **예약**

⑥ **실행 중**
객체를 만들 때 page fault → 물리 페이지 연결 = **RSS 증가**

```
$ cat /proc/<java PID>/maps (예시)
② 55e0a1200000-55e0a1201000 r-xp bin/java
⑤ 000080000000-000100000000 rw-p (heap 2GB)
⑤ 7f1c40000000-7f1c40021000 rw-p (스레드 스택)
⑤ 7f1c58000000-7f1c58800000 rwxp (Code cache)
④ 7f1c6a000000-7f1c6b200000 r-xp libjvm.so
④ 7f1c6c000000-7f1c6c1d5000 r-xp libc.so.6
③ 7ffc1a2f0000-7ffc1a311000 rw-p [stack]
```

한 줄 = 구간 하나(VMA) · 시작-끝 주소 · 권한 · 용도 · 아래로 갈수록 높은 주소

VSZ = 구간 크기의 합 · java는 수 GB

RSS = 실제로 물리 페이지가 붙은 양 · 시작 직후엔 수백 MB

비유 · 번호줄을 받고, 용도별로 구간에 이름표를 붙이고, 실제로 쓸 때만 사물함을 배정

주소 한 줄의 위 절반은 모두 같은 "커널 자리"

프로세스마다 주소 한 줄(가상 주소 공간)을 받는다 · 아래는 각자 것, 위는 공용



커널 = RAM을 관리하는 관리자

프로그램은 RAM을 직접 못 만지고 커널에게 부탁한다 (시스템 콜)

왜 위 절반에 들어 있나

부탁할 때 주소 한 줄을 바꾸지 않고 바로 커널로 넘어갈 수 있어서 빠르다

커널 메모리 맵 = 이 위 절반의 배치도

그 안의 방 중 기억할 것은 두 개 · 다음 쪽

비유 · 아래층은 각자의 집 · 꼭대기층은 관리사무소 · 모든 집 엘리베이터에 같은 관리사무소 버튼

커널 자리에서 기억할 방 두 개: 열쇠판과 카드함

커널 자리 (위 절반)

- 🔑 **직접 매핑** — Direct Mapping
- **vmalloc** — 몰라도 됨
- 🗂️ **vmemmap** — struct page 배열
- **KASAN · 커널 코드 · 모듈** — 몰라도 됨

🔑 열쇠판 = 직접 매핑

RAM 전체를 **번호 순서대로 한 줄로** 걸어 둔 곳 · RAM N번 칸 = 시작 위치 + N

언제 · 칸 안의 **내용(데이터)**을 읽고 쓸 때 · 예: 파일 내용(Page Cache)

🗂️ 카드함 = vmemmap (struct page)

RAM 칸(4KB)마다 **상태 카드 1장** · 누가 쓰나 · 최근에 썼나(LRU) · 고쳤나(dirty) · 어느 컨테이너 몫(memcg)

언제 · 칸의 **상태**를 볼 때 · 회수와 OOM 판단

예 · 메모리 부족 → 카드함에서 "오래 안 쓴 칸" 찾기(02장) → 그 칸 비우기 · 비울 게 없으면 OOM(04장)

비유 · 관리사무소 벽의 열쇠판 = 아무 사물함이나 바로 열기 · 카드함 = 사물함마다 붙은 관리 카드

버릴 수 있나, 없나

직접 쓴 메모 = 익명 메모리

힙, 스택, MAP_ANONYMOUS 매핑
예: 자바 객체, malloc 결과

세상에 하나뿐

swap 없으면 회수 불가

진짜 이름: 익명 메모리 (anon, rss)

도서관 복사본 = 파일 캐시

Page Cache, 코드, 파일 mmap
예: 로그 파일, TSDB 블록

원본이 디스크에 있음

clean은 즉시, dirty는 writeback 후 회수

진짜 이름: 파일 캐시 (file, Page Cache)

함정 · 메모리 디스크(tmpfs, emptyDir medium: Memory)는 파일처럼 보이지만 원본이 없어서 **메모처럼** 못 버린다

02 메모리 회수

free 페이지가 부족하면 커널은 죽이기 전에 회수(reclaim)부터 한다

비유 · 꽉 찬 책상 정리

이 장의 질문 · 메모리가 꽉 차면 커널은 무엇부터 할까?

회수 순서 · __alloc_pages_slowpath() | watermark min / low / high | thrashing · refault

같이 볼 문서

메모리 회수 개요 · minzkn.com/linuxkernel/pages/memory.html#reclaim-overview
Watermark 체계 · minzkn.com/linuxkernel/pages/memory.html#reclaim-watermarks
LRU Cache · minzkn.com/linuxkernel/pages/lru-cache.html
Shrinker · minzkn.com/linuxkernel/pages/shrinker.html

회수는 비용이 싼 것부터, OOM은 마지막

할당이 실패하면 __alloc_pages_slowpath()가 아래 순서로 시도하고, free 페이지가 생기면 거기서 멈춘다

1 Page Cache 회수
clean file 페이지는 바로 해제 · dirty는 writeback 후
비유 · 복사본 버리기

→

2 kswapd
free < low에서 깨어나 백그라운드 회수 · 프로세스 안 막음
비유 · 청소 도우미

→

3 direct reclaim
free < min · 할당한 프로세스가 직접 동기 회수 · 지연
비유 · 직접 치우기

→

4 compaction
페이지를 옮겨(migration) 연속 공간 확보 · 큰 order 할당용
비유 · 빈칸 모으기

→

5 swap out
anon 페이지를 swap 영역으로 · k8s 노드는 보통 swap 꺼짐
비유 · 창고로 옮기기

→

6 out_of_memory()
회수 실패 · victim 선택 후 SIGKILL
비유 · 강제 퇴장

비용 낮음 · 지연 없음 — 비용 높음 · 지연 또는 프로세스 종료

함수 순서 · ① wake_all_kswapds() ② direct reclaim (try_to_free_pages()) ③ try_to_compact_pages() ④ out_of_memory()

이름을 풀면 하는 일이 보인다

용어	원래 뜻 (단어 풀이)	커널에서 하는 일 (원문 기준)	비유
Reclaim 회수	re + claim "도로 되찾다"	빌려준 페이지를 되찾아 free 페이지로 되돌림 기준은 LRU 대상 2종: file은 원본에서 다시 읽으면 되고, anon은 swap으로 내보내야 함 경로 try_to_free_pages() → shrink_node()	책상 정리
LRU 교체 정책	Least Recently Used "가장 오래 안 쓴 것"	오래 안 쓴 페이지는 앞으로도 안 쓸 거라고 가정 · lruvec에 리스트 5개 INACTIVE/ACTIVE × ANON/FILE + UNEVICTABLE 새 페이지는 inactive · 두 번 접근하면 active 승격 · 6.1+ MGLRU 세대 2~4개	오래 안 본 종이부터
Page Cache 페이지 캐시	cache ← 프랑스어 cacher "숨겨 두다"	파일 내용을 담은 페이지 · clean은 바로, dirty는 writeback 후 회수 회수된 자리에 shadow entry를 남겨, 금방 다시 읽히면(refault) 바로 active로	도서관 복사본
kswapd 회수 스레드	kernel swap daemon daemon = 뒤에서 일하는 프로세스	NUMA 노드마다 1개 · free가 low 아래면 깨어나 high까지 회수 프로세스를 막지 않음 · 지표 pgscan_kswapd, pgsteal_kswapd	청소 도우미
Direct reclaim 직접 회수	direct "직접, 몸소"	free가 min 아래면 할당 경로에서 직접 회수 프로세스가 막힘 = 지연(latency) 스파이크 · 지표 allocstall_normal, pgsteal_direct	하던 일 멈추고 치움

기준선과 마지막 수단들

용어	원래 뜻 (단어 풀이)	커널에서 하는 일 (원문 기준)	비유
Watermark 워터마크	배에 그은 물높이 선 "수위선"	zone마다 기준선 3개 · high 위면 kswapd 수면 low 아래면 kswapd 깨움 · min 아래면 direct reclaim · 실제 값은 /proc/zoneinfo	눈금선 3개
Swap 스왑	swap "맞바꾸다"	anon 페이지를 디스크 swap 영역으로 내보내고 자리 비움 vm.swappiness 0~200, 기본 60 · 높을수록 anon을 더 적극적으로 회수 swap이 없으면 anon은 버릴 곳이 없어 결국 OOM	창고로 옮기기
Shrinker 캐시 수축기	shrink "줄어들게 하다"	페이지가 아닌 커널 캐시(dentry, inode, 파일시스템 메타데이터)를 회수하는 콜백 register_shrinker()로 등록 · 회수할 때 함께 불림	서랍 속 메모지 정리
Compaction 메모리 압축	compact "꽉 눌러 다지다"	흩어진 빈 페이지를 이어 붙여 연속 공간 확보 (큰 order 할당용) 스캐너 2개: migration은 zone 앞에서, free는 zone 끝에서 · 5.9+ proactive 지표 compact_stall, compact_success, compact_fail	빈칸 모으기
Thrashing 스래싱	thrash "몸부림치다"	버린 페이지를 곧바로 다시 읽기 반복 · 회수만 하고 일은 못 함 /proc/pressure/memory (PSI, 4.20+) · some 일부 대기, full 전부 대기	버리고 또 복사

watermark: free 페이지 수가 회수 방식을 정한다

커널은 zone마다 기준선 3개 를 놓고 어느 선 아래냐에 따라 누가, 어떻게 회수할지가 바뀐다

high **free ≥ high**	회수 중단 · kswapd sleep
low **free < low**	kswapd wake · high까지 **비동기** 회수 · 프로세스 안 막음
min **free < min**	direct reclaim · 할당 경로에서 **동기** 회수 · 지연 · 실패하면 OOM

비유 · 남은 공간에 그어 둔 눈금선 3개

노드 전체 기준

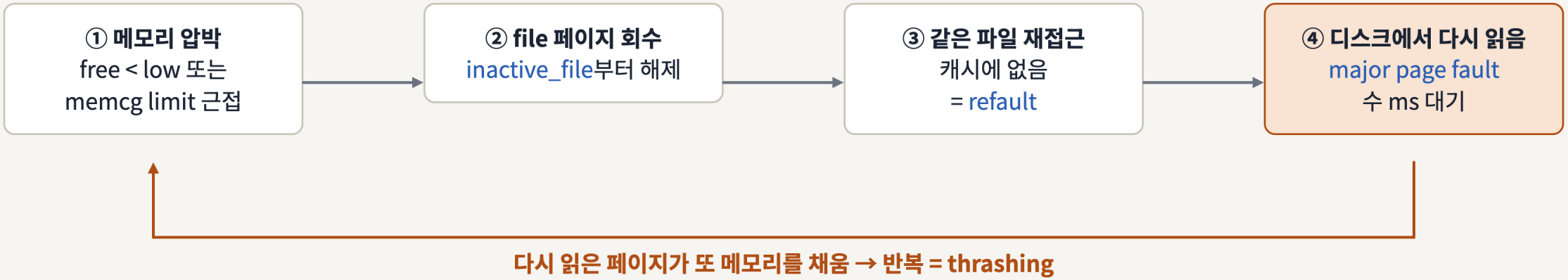
zone 단위 free 페이지 · 값은 /proc/zoneinfo · 설정 vm.min_free_kbytes, vm.watermark_scale_factor

Pod OOMKilled는 대부분 별개

노드 watermark가 아니라 cgroup memory.max 초과 = memcg OOM · 23쪽

thrashing: 회수한 페이지가 곧바로 다시 필요해지는 상태

OOM은 나지 않지만 CPU와 디스크가 회수와 재로딩에 묶여 처리가 느려진다



커널이 감지하는 방법	원리
workingset 감지	회수한 자리에 shadow entry를 남김 · 다시 읽힐 때 refault distance가 작으면 바로 active로 승격
PSI	/proc/pressure/memory · 메모리를 기다리며 멈춘 시간 비율 · some 일부 대기, full 전부 대기
major fault	디스크 I/O가 필요한 page fault 횟수 · /proc/vmstat pgmajfault

비유 · 버린 복사본을 또 도서관에서 복사해 오는 일의 반복

03 Page Cache 개요

파일 내용을 메모리에 두는 커널 캐시 · 원본이 디스크에 있어서 회수 1순위

비유 · 도서관 책 복사본

이 장의 질문 · 메모리 사용량에 섞인 Page Cache는 OOM과 어떤 관계일까?

read/write 경로 · clean/dirty/writeback

memory.stat 구성 · anon/active_file/inactive_file

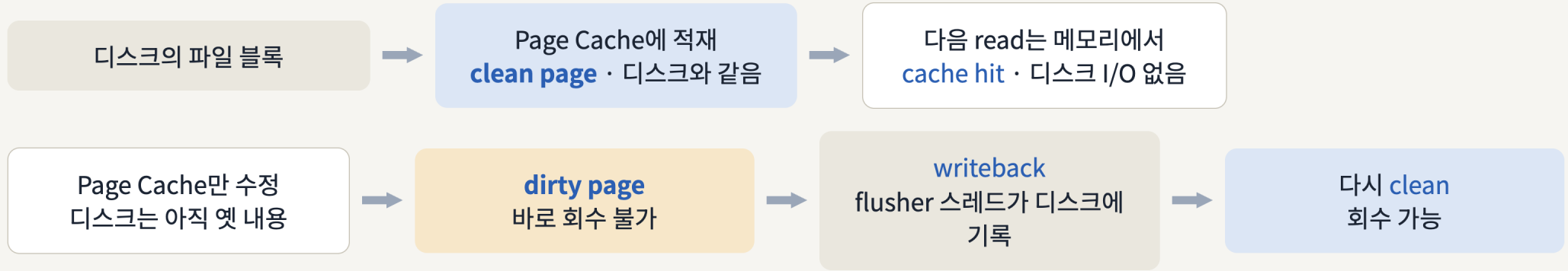
memcg limit에서의 회수와 OOM

같이 볼 문서

파일 기반 페이지 (Page Cache) · minzkn.com/linuxkernel/pages/memory.html#anonymous-vs-file-pages
active / inactive LRU · minzkn.com/linuxkernel/pages/lru-cache.html
/proc 메모리 인터페이스 · minzkn.com/linuxkernel/pages/memory.html#proc-interfaces
Memory Cgroup (memory.stat) · minzkn.com/linuxkernel/pages/memcg.html

Page Cache: 파일 I/O는 메모리를 거친다

메모리 접근이 디스크보다 대략 1000배 이상 빠르기 때문에, 커널은 남는 메모리를 파일 캐시로 쓴다



비유 · 한 번 읽은 책을 책상에 복사해 두기 · 고친 복사본은 원본에 옮겨 적어야 버릴 수 있음

MemFree vs MemAvailable

/proc/meminfo · Free는 완전히 빈 페이지, Available은 Free + 회수 가능한 캐시를 더한 추정치

회수 순서 · LRU

inactive_file부터 회수 · active_file은 inactive로 내려간 뒤 회수 · 두 번 접근하면 active로 승격

컨테이너 메모리는 세 부분으로 나뉜다

cgroup memory.stat의 항목 기준 · 회수할 수 있는 부분과 없는 부분이 다르다

anon	active_file	inactive_file
힙·스택 · swap 없으면 회수 불가	최근 쓴 Page Cache	오래 안 쓴 Page Cache · 회수 1순위

rss ≈ anon

working_set = usage − inactive_file

usage = memory.current · cgroup에 charge된 전부

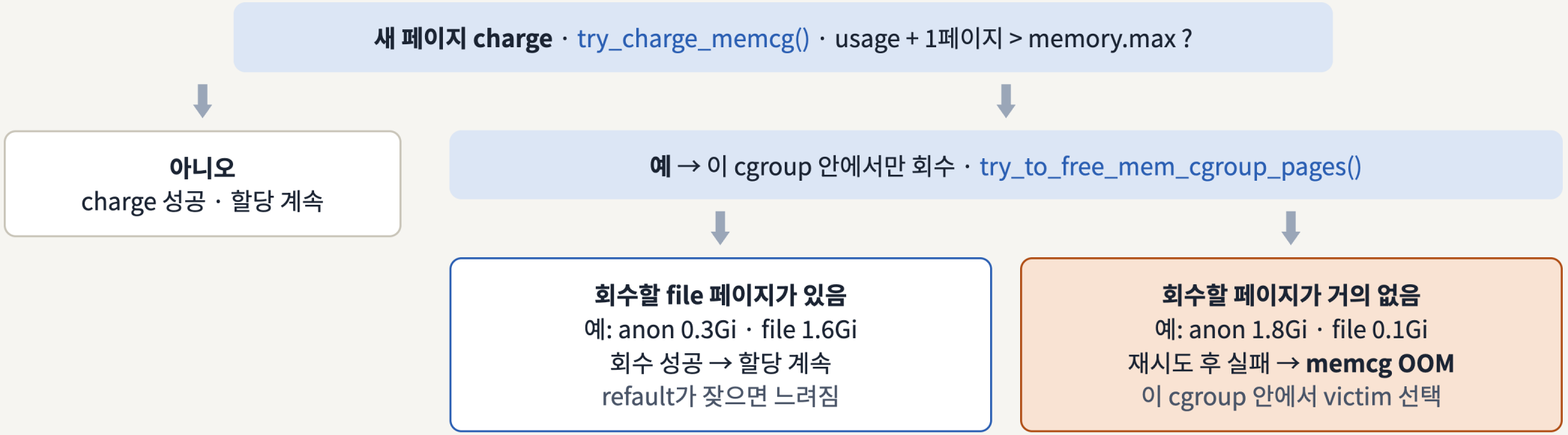
비유 · 책상 위 종이 = 직접 쓴 메모(anon) + 자주 보는 복사본 + 한동안 안 본 복사본

working_set · kubelet이 eviction 판단과 kubectl top에 쓰는 값

anon이 memory.max에 닿으면 · 회수할 페이지가 없어 memcg OOM

memcg limit에 닿으면: 먼저 회수, 안 되면 OOM

예시 · memory.max 2Gi 컨테이너가 새 페이지를 할당할 때



비유 · 책상이 꽉 차면 먼저 복사본을 버려 보고, 버릴 게 없을 때만 퇴장

04 OOM 트리거 경로

비유: 쫓아내기가 시작될 때

이 장의 질문 · 언제, 어떤 경로로 OOM Killer가 불려 올까?

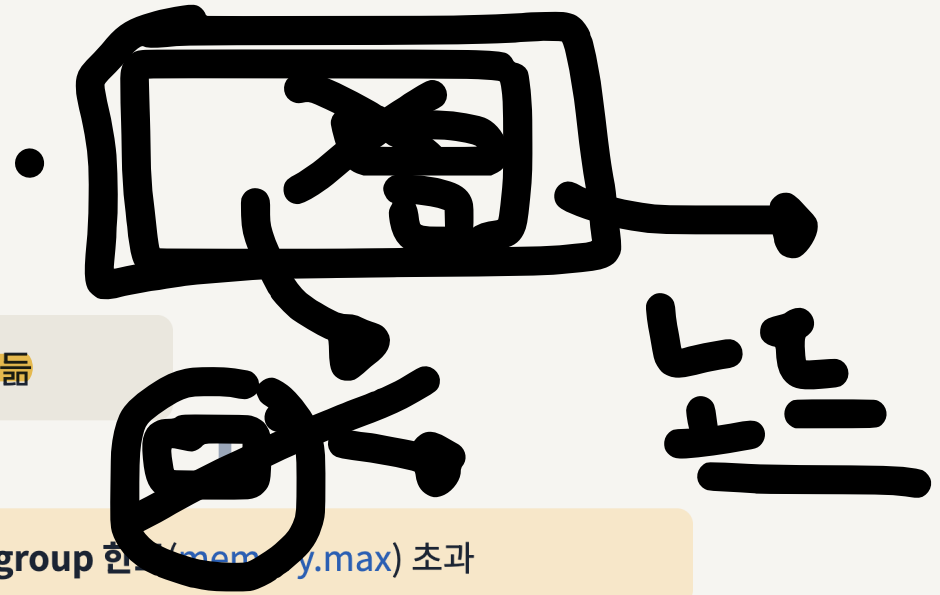
두 갈래 · 전역 vs memcg | 전역 OOM 함수 경로 | 두 길 비교표 | OOM 범위 · constraint

같이 볼 문서

OOM Killer · 트리거 경로 · minzkn.com/linuxkernel/pages/oom-killer.html
OOM Killer 개요 · minzkn.com/linuxkernel/pages/memory.html#oom-overview
Memory Cgroup (memcg OOM) · minzkn.com/linuxkernel/pages/memcg.html
cgroup v2 · memory.max, memory.events · docs.kernel.org/admin-guide/cgroup-v2.html

04 · OOM 트리거 경로 · 개념

OOM이 일어나는 범위는 두 가지



페이지 할당 요청 → 회수로도 공간을 못 만듦

노드 전체(노드 RAM) free 페이지 고갈

노드의 모든 프로세스가 후보
다른 Pod의 프로세스도 죽을 수 있음

전역 OOM · CONSTRAINT_NONE · 비유: 방 전체가 꽉 참

cgroup 한도(memory.max) 초과

같은 cgroup의 프로세스만 후보
노드에 free 메모리가 남아도 발생

memcg OOM · CONSTRAINT_MEMCG · 비유: 내 책상만 넘침

노드에 메모리가 남아 있어도 cgroup 한도를 넘으면 OOM · k8s의 OOMKilled는 대부분 memcg OOM

04 · OOM 트리거 경로 · 전역 OOM 경로

전역 OOM: 할당 실패에서 SIGKILL까지

1

fast path 실패

free가 watermark 아래라 freelist에서 할당 실패

`alloc_pages()`

비유 · 빠른 창구에서 "빈 사물함 없음"

2

slowpath 재시도

direct reclaim, compaction 재시도

`__alloc_pages_slowpath()`

비유 · 느린 창구에서 정리하며 다시 시도

3

OOM 진입

재시도 한도 초과 · __GFP_NORETRY 요청은 여기서 포기

`__alloc_pages_may_oom()`

비유 · 정리해도 안 돼 퇴장 절차 시작 · "없으면 말고" 신청은 포기

4

직렬화

mutex 획득 후 할당 한 번 더 시도 · 동시 kill 방지

`oom_lock`

비유 · 퇴장 결정 열쇠는 하나 · 잡은 창구만 진행

5

constraint 판별

전역 / cpuset / mempolicy / memcg

`out_of_memory()`

비유 · 건물 전체인가, 한 주민의 한도인가

6

victim 선택

후보 전체 중 oom_badness() 최고점

`select_bad_process()`

비유 · 후보를 훑으며 벌점 최고점 찾기

7

강제 퇴장

dmesg에 기록 후 SIGKILL

`oom_kill_process()`

비유 · 사건 기록부에 적고 즉시 퇴장 명령

8

메모리 해제

victim의 anon 메모리를 비동기로 해제

`oom_reaper`

비유 · 나가기 전에 짐(anon)부터 먼저 치움

04 · OOM 트리거 경로 · 두 길 비교

전역 OOM vs memcg OOM

구분	전역 OOM	memcg OOM
왜	노드 RAM 전체가 바닥	cgroup memory.max 초과
누가 후보	노드의 모든 프로세스	같은 cgroup 안의 프로세스만
점수 보정 기준	totalpages = 노드 RAM	totalpages = memory.max
dmesg constraint	CONSTRAINT_NONE	CONSTRAINT_MEMCG
관측	dmesg	memory.events의 oom_kill
k8s에서 빈도	가끔	대부분 이쪽

memory.oom.group · 1이면 victim 하나가 아니라 cgroup의 모든 프로세스를 함께 kill

비유 · 전역 OOM = 방 전체가 꽉 참 · memcg OOM = 내 책상만 넘침

★ 발표 제목의 답이 나오는 장

05 oom_badness() 스코어링

oom_badness()가 후보마다 점수를 매기고, 최고점 프로세스를 kill한다

비유 · 벌점이 가장 높은 사람이 퇴장

이 장의 질문 · OOM Killer는 어떤 기준으로 프로세스를 고를까?

점수 공식 · oom_badness()

보정값 · oom_score_adj −1000~1000

totalpages · 전역과 memcg에서 다름

같이 볼 문서

OOM Killer · oom_badness() 스코어링 · minzkn.com/linuxkernel/pages/oom-killer.html

OOM 점수 계산 · minzkn.com/linuxkernel/pages/memory.html#oom-scoring

/proc/<pid>/oom_score, oom_score_adj · docs.kernel.org/filesystems/proc.html

oom_badness() = 실제 점유 페이지 + oom_score_adj 보정

기본 점수

프로세스가 실제로 점유한 페이지 수

RSS(anon+file+shmem) + swap + 페이지 테이블

+

보정 점수 · oom_score_adj

−1000 ~ +1000 사이 값

adj × totalpages / 1000 페이지를 더함 · 1 = 0.1%

=

최종 점수 · oom_score

최고점 프로세스가 victim

최소 1점

안 세는 것

VSZ(가상 크기) · 예약만 크게 해도 점수에 안 들어감

면제

adj −1000, 커널 스레드, PID 1(init)

주의

후보가 없으면 kernel panic · −1000 남발 금지

비유 · 벌점 = 책상 위에 실제로 둔 양 + 등급에 따른 보정

확인 · cat /proc/<pid>/oom_score /proc/<pid>/oom_score_adj · 소스 mm/oom_kill.c

05 · oom_badness() 스코어링 · 보정 점수

oom_score_adj: 점수에 얹는 가중치

점수에 **adj × totalpages ÷ 1000** 페이지를 더한다 · adj 1 = 전체 메모리의 0.1%

값이 낮을수록 보호 · 높을수록 먼저 kill

`-1000`	**면제** 후보에서 완전히 제외 · 남용하면 kernel panic
`-900 ~ -500`	**강한 보호** sshd, DB 같은 필수 프로세스
`0`	**기본값** 보정 없음 · 대부분의 프로세스
`+500`	**우선 대상** 배치 작업처럼 먼저 죽어도 되는 것
	거의 첫 번째

계산 예 · 32GiB 노드의 전역 OOM (점수를 GiB로 환산)

프로세스	RSS	adj	보정	점수
java-app	12.0	0	0	12.0
batch-job	2.0	+500	+16.0	18.0 · victim
postgres	6.0	−900	−28.8	−22.8 → 최소 1
sshd	0.1	−1000	면제 · 후보 아님	

batch-job은 RSS가 가장 작아도 **보정 +16** 때문에 점수 1위

같은 adj+500도 범위에 따라 무게가 다르다

totalpages = 전역 OOM이면 노드 메모리(RAM+swap) · memcg OOM이면 memory.max
32GiB 노드 → **+16GiB** · limit 2GiB cgroup → **+1GiB**

누가, 어떻게 정하나

echo 500 > /proc/<pid>/oom_score_adj · systemd OOMScoreAdjust=
k8s에서는 kubelet이 QoS class로 자동 설정

비유 · 벌점에 얹는 가산점 · 경비 명단에서 빼 주는 면제권

06 OOM dmesg 메시지 해부

OOM kill 직후 커널이 dmesg에 남기는 판단 근거를 읽는다

비유 · 사건 기록부

이 장의 질문 · OOM이 나면 무엇을 보고 원인을 찾을까?

dmesg 다섯 구역 | 읽는 순서 · 질문 5개 | 부록 · 장애 대응 순서 | 부록 · 명령 모음

같이 볼 문서

OOM Killer · dmesg 메시지 해부 · minzkn.com/linuxkernel/pages/oom-killer.html
메모리 운영 플레이북 · minzkn.com/linuxkernel/pages/memory.html#memory-playbook
OOM Killer 오동작 사례 · minzkn.com/linuxkernel/pages/memory.html#oom-killer-bugs

06 · OOM dmesg 메시지 해부

OOM 로그는 다섯 부분으로 나뉜다

```
① java invoked oom-killer: gfp_mask=0xcc0(GFP_KERNEL), order=0, oom_score_adj=937
CPU: 3 PID: 4821 Comm: java Not tainted 6.1.x
② memory: usage 2097152kB, limit 2097152kB, failcnt 1843
anon 2100449280 file 18771968 shmem 0 ...
③ [ pid ] uid tgid total_vm rss pgtables_bytes swapents oom_score_adj name
[ 4821] 1000 4821 1458026 517388 4620288 0 937 java
④ oom-kill:constraint=CONSTRAINT_MEMCG,...,oom_memcg=/kubepods.slice/...,task=java,pid=4821
⑤ Memory cgroup out of memory: Killed process 4821 (java) total-vm:5832104kB,
anon-rss:2051220kB, file-rss:18332kB, shmem-rss:0kB, UID:1000 pgtables:4512kB
```

① 할당 요청자 · invoked oom-killer
② memcg 사용량과 한도 · memcg usage / limit
③ 후보 프로세스 목록 · task dump
④ OOM 범위 · constraint
⑤ victim 정보 · Killed process

비유 · 사건 기록부 한 장

06 · OOM dmesg 메시지 해부 · 읽는 순서

위에서부터 말고, 질문 순서대로

칸	질문	앞 쪽 예시의 답
④	전역 OOM인가, memcg OOM인가?	CONSTRAINT_MEMCG · cgroup 한도 초과
①	요청 크기는? (order, 0이면 4KB 한 장)	order=0 · 작은 요청, 단순히 모자람
②	memcg 한도와 사용량은? (limit, usage)	둘 다 2GiB · 꽉 참
⑤	anon이 많았나, file이 많았나? (anon-rss vs file-rss)	anon-rss 약 2.0GB · 거의 다 anon
③	다른 프로세스도 많이 썼나? (task 목록 rss)	java 하나가 거의 다 차지

결론 · java의 anon 페이지가 memory.max를 채움 · 회수할 file 페이지가 없어 memcg OOM

CONSTRAINT_NONE이었다면 · 노드 전체 메모리 고갈 · 노드의 모든 프로세스가 후보

`journalctl -k | grep -iE 'oom|Killed process'` · `kubectl describe pod` → `Last State: OOMKilled (137)`

기억할 세 가지

1. 점수는 VSZ가 아니라 **실제 점유 페이지(RSS + swap + 페이지 테이블)**로 매긴다
2. Page Cache는 회수되지만, **swap이 없으면 anon 페이지는 회수할 수 없다**
3. victim은 **oom_badness()** 최고점 · 점수 = 점유 페이지 + **oom_score_adj** 보정

범위는 dmesg의 constraint로 구분 · CONSTRAINT_NONE = 노드 전체 · CONSTRAINT_MEMCG = memory.max 초과